

T-Sharp (T#) — Şifreleme ve Hash

Başlamadan Önce — Modül Aktivasyonu

T#'da şifreleme ve hash işlemleri **ayrı modüller** olarak tasarlanmıştır. Kullanmadan önce dosyanın en başında aktive edilmeleri gerekir.

```
kullan sifrele    // Base64, XOR, dosya şifreleme için
kullan hash       // SHA256, SHA512, MD5, SHA1, HMAC için
```

İkisini aynı anda da kullanabilirsiniz:

```
kullan sifrele
kullan hash
```

İçindekiler

Şifreleme Modülü (`kullan sifrele`)

1. Base64 Şifreleme — `b64_sifrele`
2. Base64 Çözme — `b64_coz`
3. XOR Şifreleme — `xor_sifrele`
4. XOR Çözme — `xor_coz`
5. Dosya Şifreleme — `dosya_sifrele`
6. Dosya Çözme — `dosya_coz`
7. Şifreli Dosyaya Yazma — `sifreli_yaz`
8. Şifreli Dosya Okuma — `sifreli_oku`

Hash Modülü (`kullan hash`)

1. SHA256 — `sha256`
2. SHA512 — `sha512`
3. MD5 — `md5`
4. SHA1 — `sha1`
5. Dosya SHA256 — `dosya_sha256`
6. HMAC — `hmac_hesapla`
7. Hash Doğrulama — `hash_dogrula`
8. Tam Örnek Programlar

ŞİFRELEME MODÜLÜ

1. Base64 Şifreleme — `b64_sifrele`

Metni Base64 formatına kodlar. Veriyi gizlemez, sadece taşınabilir bir formata çevirir. Genellikle veri iletimi için kullanılır.

Söz dizimi

```
b64_sifrele <degisken> <metin>
```

Örnek

```
kullan sifrele

b64_sifrele kodlanmis "Merhaba T-Sharp!"
yazdır kodlanmis
// TWhocmhYmEgVC1TaGFycCE= (Değişebilir)
```

2. Base64 Çözme — `b64_coz`

Base64 kodlu metni orijinal haline döndürür.

Söz dizimi

```
b64_coz <degisken> <b64_metin>
```

Örnek

```
kullan sifrele

b64_sifrele kodlanmis "Gizli mesaj"
yazdır "Şifreli:" kodlanmis

b64_coz acik kodlanmis
yazdır "Açık      :" acik
// Açık      : Gizli mesaj
```

3. XOR Şifreleme — `xor_sifrele`

Metni bir anahtar ile XOR işlemine tabi tutar, ardından Base64 ile kodlar. Aynı anahtar olmadan çözülemez. Hafif ve hızlı bir şifreleme yöntemidir.

Söz dizimi

```
xor_sifrele <degisken> <metin> <anahtar>
```

Örnek

```
kullan sifrele

degisken mesaj    = "Bu cok gizli bir mesajdir"
degisken anahtar  = "benim_super_anahtarim"

xor_sifrele sifreli mesaj anahtar
yazdır "Şifreli:" sifreli
```

Önemli: Anahtar ne kadar uzun ve karmaşık olursa şifreleme o kadar güçlü olur. Anahtar boş bırakılamaz — hata verir.

4. XOR Çözme — `xor_coz`

`xor_sifrele` ile şifrelenmiş metni, **aynı anahtarla** çözer.

Söz dizimi

```
xor_coz <degisken> <sifreli_metin> <anahtar>
```

Örnek

```
kullan sifrele

degisken mesaj    = "Raspberry Pi ile T-Sharp!"
degisken anahtar  = "gizli_anahtar_42"

// Şifrele
xor_sifrele sifreli mesaj anahtar
yazdır "Şifreli :" sifreli

// Çöz
xor_coz cozulmus sifreli anahtar
yazdır "Çözülmüş:" cozulmus
```

Çıktı:

```
Şifreli : Hg8cFRUKEhMdGQYGCxYLBxkFGg== (Değişebilir)
Çözülmüş: Raspberry Pi ile T-Sharp!
```

5. Dosya Şifreleme — `dosya_sifrele`

Bir dosyayı XOR + Base64 ile şifreleyerek yeni bir `.enc` dosyasına kaydeder.

Söz dizimi

```
dosya_sifrele "<kaynak_dosya>" "<hedef_dosya>" "<anahtar>"
```

Örnek

```
kullan sifrele

// Önce örnek bir dosya oluştur
dosya_yaz "belge.txt" "Bu dosyanın icindekiler gizlidir."

// Şifrele
dosya_sifrele "belge.txt" "belge.enc" "super_gizli_sifre"
yazdır "Dosya şifrelendi!"
```

6. Dosya Çözme — dosya_coz

`dosya_sifrele` ile şifrelenmiş bir dosyayı, **aynı anahtarla** çözer.

Söz dizimi

```
dosya_coz "<sifreli_dosya>" "<hedef_dosya>" "<anahtar>"
```

Örnek

```
kullan sifrele

// Şifreli dosyayı çöz
dosya_coz "belge.enc" "belge_acik.txt" "super_gizli_sifre"

// Çözülmüş dosyayı oku ve yazdır
degisken icerik = dosya_oku("belge_acik.txt")
yazdır icerik
```

7. Şifreli Dosyaya Yazma — sifreli_yaz

Bir metni şifreleyerek doğrudan dosyaya yazar. Ara adım (önce yaz, sonra şifrele) gerektirmez.

Söz dizimi

```
sifreli_yaz "<dosya_yolu>" "<icerik>" "<anahtar>"
```

Örnek

```
kullan sifrele
```

```
sifreli_yaz "gizli_notlar.enc" "Banka hesabı şifresi: 1234" "ana_sifre"
yazdır "Gizli not şifreli olarak kaydedildi."
```

8. Şifreli Dosya Okuma — `sifreli_oku`

`sifreli_yaz` ile yazılmış bir dosyayı okuyup çözer ve değişkene atar.

Söz dizimi

```
sifreli_oku <degisken> "<dosya_yolu>" "<anahtar>"
```

Örnek

```
kullan sifrele
```

```
// Yaz
sifreli_yaz "not.enc" "Bu cok ozel bir not." "sifrem123"

// Oku
sifreli_oku icerik "not.enc" "sifrem123"
yazdır icerik
// Bu cok ozel bir not.
```

HASH MODÜLÜ

Hash fonksiyonları **tek yönlüdür** — bir metinden hash üretilir ama hash'ten metin geri alınamaz. Şifre doğrulama, dosya bütünlük kontrolü ve veri imzalama için kullanılır.

9. SHA256 — sha256

En yaygın kullanılan güvenli hash algoritması. Şifre saklama ve doğrulama için önerilir.

Söz dizimi

```
sha256 <degisken> <metin>
```

Örnek

```
kullan hash

sha256 sifre_hash "kullanici_sifresi_123"
yazdır sifre_hash
// e3b0c44298fc1c149afb... (64 karakter hex)
```

10. SHA512 — sha512

SHA256'dan daha uzun (128 karakter) ve daha güçlü hash üretir.

Söz dizimi

```
sha512 <degisken> <metin>
```

Örnek

```
kullan hash

sha512 guclu_hash "onemli_veri"
yazdır guclu_hash
```

11. MD5 — md5

Hızlı ama **güvenlik açısından zayıf** bir hash algoritması. Dosya bütünlük kontrolü gibi güvenlik gerektirmeyen durumlarda kullanılabilir. Şifre saklamak için **kullanmayın**.

Söz dizimi

```
md5 <degisken> <metin>
```

Örnek

```
kullan hash  
  
md5 kontrol_hash "dosya_icerigi_buraya"  
yazdır kontrol_hash
```

12. SHA1 — sha1

MD5'ten biraz daha güvenli ama SHA256 kadar güçlü değil. Eski sistemlerle uyumluluk için kullanılır.

Söz dizimi

```
sha1 <degisken> <metin>
```

Örnek

```
kullan hash  
  
sha1 eski_hash "eski_sistem_verisi"  
yazdır eski_hash
```

13. Dosya SHA256 — dosya_sha256

Bir dosyanın SHA256 hash'ini hesaplar. Dosyanın bozulup bozulmadığını veya değiştirilip değiştirilmediğini kontrol etmek için kullanılır. Büyük dosyaları 64KB'lık parçalar halinde işler.

Söz dizimi

```
dosya_sha256 <degisken> "<dosya_yolu>"
```


Örnek

```
kullan hash
```

```
dosya_sha256 dosya_hash "program.exe"  
yazdır "Dosya hash:" dosya_hash
```

```
kullan hash
```

```
// Dosya bütünlük doğrulaması  
degisken beklenen = "a3f5d2e1c9b8..." // daha önce kaydedilmiş hash  
  
dosya_sha256 mevcut_hash "indirilen_dosya.zip"  
  
eger mevcut_hash esittir beklenen:  
    yazdır "Dosya bütünlüğü doğrulandı. Güvenli!"  
degilse:  
    yazdır "UYARI: Dosya değiştirilmiş olabilir!"  
son
```

14. HMAC — `hmac_hesapla`

Bir mesajın hem **bütünlüğünü** hem de **kaynağını** doğrulamak için kullanılır. Sadece hash değil, aynı zamanda gizli anahtar gerektirdiği için daha güvenlidir.

Söz dizimi

```
hmac_hesapla <degisken> "<anahtar>" "<metin>"
```

Örnek

```
kullan hash
```

```
hmac_hesapla imza "gizli_api_anahtari" "gonderilecek_mesaj"  
yazdır "HMAC İmzası:" imza
```

```
kullan hash
```

```
// API isteğini imzalama
degisken api_key = "abc123secret"
degisken mesaj   = "kullanici_id=42&islem=odeme&tutar=100"

hmac_hesapla istek_imzasi api_key mesaj
yazdır "İstek imzası:" istek_imzasi
```

15. Hash Doğrulama — `hash_dogrula`

Bir metnin hash'ini hesaplayıp beklenen değerle karşılaştırır. **Sabit zamanlı karşılaştırma** (timing attack koruması) kullanır — güvenli şifre doğrulaması için idealdir.

Söz dizimi

```
hash_dogrula <degisken> <metin> <beklenen_hash> ["algoritma"]
```

Algoritma parametresi: `sha256` (varsayılan), `sha512`, `md5`, `sha1`

Örnek

```
kullan hash

// Şifreyi hashle ve kaydet
sha256 kayitli_hash "kullanici_sifresi"

// Sonra doğrula
hash_dogrula sonuc "kullanici_sifresi" kayitli_hash "sha256"

eger sonuc:
    yazdır "Şifre doğru! Giriş başarılı."
degilse:
    yazdır "Hata: Yanlış şifre."
son
```

kullan hash

```
// SHA512 ile doğrulama
sha512 guclu_hash "super_gizli_sifre"

hash_dogrula kontrol "super_gizli_sifre" guclu_hash "sha512"
yazdır kontrol    // dogru

hash_dogrula kontrol "yanlis_sifre" guclu_hash "sha512"
yazdır kontrol    // yanlis
```

16. Tam Örnek Programlar

Örnek 1 — Şifreli Not Defteri

```
kullan sifrele

degisken dosya = "notlar.enc"

girdi ana_sifre "Ana şifrenizi girin: "

dongu dogru:
    yazdır "=== GİZLİ NOT DEFTERİ ==="
    yazdır "1) Not ekle"
    yazdır "2) Notları görüntüle"
    yazdır "3) Çıkış"
    girdi secim "Seçim: "

    eger secim esittir "1":
        girdi yeni_not "Notunuz: "

        // Mevcut notları oku (varsa)
        eger dosya_var_mi(dosya):
            sifreli_oku eski_notlar dosya ana_sifre
            degisken tum_notlar = eski_notlar + "\n" + yeni_not
        degilse:
            degisken tum_notlar = yeni_not
        son

        sifreli_yaz dosya tum_notlar ana_sifre
        yazdır "Not kaydedildi!"
    son

    eger secim esittir "2":
        eger dosya_var_mi(dosya):
            sifreli_oku notlar dosya ana_sifre
            eger notlar degildir hic:
                yazdır "--- Notlarınız ---"
                yazdır notlar
            degilse:
                yazdır "Şifre yanlış veya dosya bozuk."
            son
        degilse:
```

```
yazdır "Henüz not yok."
```

```
son
```

```
son
```

```
eger secim esittir "3":
```

```
yazdır "Güle güle!"
```

```
dur
```

```
son
```

```
son
```

Örnek 2 — Kullanıcı Kayıt ve Giriş Sistemi

```
kullan hash

degisken kullanıcı_dosyasi = "kullanıcılar.txt"

fonksiyon kayıt_ol(kullanıcı_adi, şifre):
    // Şifreyi hashle - asla düz metin saklanmaz!
    sha256 şifre_hash şifre
    degisken kayıt = kullanıcı_adi + ":" + şifre_hash
    dosya_ekle kullanıcı_dosyasi kayıt + "\n"
    yazdır "Kayıt başarılı:" kullanıcı_adi
son

fonksiyon giriş_yap(kullanıcı_adi, şifre):
    eger değil dosya_var_mi(kullanıcı_dosyasi):
        yazdır "Kullanıcı bulunamadı."
        dondur yanlış
    son

    degisken icerik = dosya_oku(kullanıcı_dosyasi)
    liste satirlar = bol(icerik, "\n")
    sha256 girilen_hash şifre

    her satir icinde satirlar:
        eger uzunluk(satir) buyuktur 0:
            liste parcalar = bol(satir, ":")
            eger parcalar[0] esittir kullanıcı_adi:
                hash_dogrula eslesme şifre parcalar[1] "sha256"
                eger eslesme:
                    yazdır "Giriş başarılı! Hoş geldin," kullanıcı_adi
                    dondur dogru
                değilse:
                    yazdır "Hata: Yanlış şifre."
                    dondur yanlış
            son
        son
    son

    yazdır "Kullanıcı bulunamadı:" kullanıcı_adi
    dondur yanlış

son
```

```
// Kullanım

kayit_ol("talha", "guvenli_sifre_123")
kayit_ol("ayse", "baska_sifre_456")


giris_yap("talha", "guvenli_sifre_123") // Başarılı
giris_yap("talha", "yanlis_sifre")      // Hata
giris_yap("olmayan", "herhangi")        // Bulunamadı
```

Örnek 3 — Dosya Bütünlük Denetleyicisi

kullan hash

```
// Kritik dosyaların hash değerlerini kaydet
liste kritik_dosyalar ["ana.tsharp", "ayarlar.txt", "veri.csv"]
degisken hash_dosyasi = "dosya_hashleri.txt"
```

```
fonksiyon hashleri_kaydet():
    dosya_yaz hash_dosyasi ""
    her dosya icinde kritik_dosyalar:
        eger dosya_var_mi(dosya):
            dosya_sha256 h dosya
            dosya_ekle hash_dosyasi dosya + ":" + h + "\n"
            yazdır "Hash kaydedildi:" dosya
        degilse:
            yazdır "Dosya bulunamadi:" dosya
    son
son
yazdır "Tüm hashler kaydedildi."
```

son

```
fonksiyon hashleri_dogrula():
    eger degil dosya_var_mi(hash_dosyasi):
        yazdır "Hash dosyası bulunamadı. Önce kaydet."
        dondur
    son
```

```
degisken kayitlar = dosya_oku(hash_dosyasi)
liste satirlar = bol(kayitlar, "\n")
degisken temiz = 0
degisken degismis = 0
```

```
her satir icinde satirlar:
    eger uzunluk(satir) buyuktur 0:
        liste parcalar = bol(satir, ":")
        degisken dosya_adi = parcalar[0]
        degisken kayitli_h = parcalar[1]

        eger dosya_var_mi(dosya_adi):
            dosya_sha256 mevcut_h dosya_adi
            eger mevcut_h esittir kayitli_h:
                yazdır "Temiz:" dosya_adi
```



```
        temiz += 1
    degilse:
        yazdır "DEĞİSTİRİLMİŞ:" dosya_adi
        degismis += 1
    son
degilse:
    yazdır "SİLİNMİŞ:" dosya_adi
    degismis += 1
son
son

yazdır "--- Sonuç ---"
yazdır "Temiz      :" temiz
yazdır "Degismis  :" degismis
son

// Kullanım
hashleri_kaydet()
yazdır
hashleri_dogrula()
```

Örnek 4 — XOR ile Mesajlaşma Simülasyonu

```
kullan sifrele

degisken paylasilan_anahtar = "T_Sharp_Gizli_Anahtar_2025"

// Gönderici tarafı
degisken mesaj = "Bulusma yeri: Kizilayda saat 15:00"
yazdır "Orijinal mesaj:" mesaj

xor_sifrele sifreli_mesaj mesaj paylasilan_anahtar
yazdır "Şifreli (iletilen):" sifreli_mesaj
yazdır

// Alıcı tarafı (aynı anahtarla çözer)
xor_coz alinan_mesaj sifreli_mesaj paylasilan_anahtar
yazdır "Çözülen mesaj:" alinan_mesaj

// Doğrulama
eger alinan_mesaj esittir mesaj:
    yazdır "Mesaj bütünlüğü doğrulandı."
son
```

Algoritma Karşılaştırma Tablosu

Algoritma	Komut	Çıktı Uzunluğu	Güvenlik	Kullanım Amacı
Base64	b64_sifrele	~%33 büyür	Yok (kodlama)	Veri taşıma
XOR	xor_sifrele	Base64 kadar	Orta	Hafif şifreleme
MD5	md5	32 karakter	Düşük ⚠	Sadece kontrol
SHA1	sha1	40 karakter	Düşük ⚠	Eski sistemler
SHA256	sha256	64 karakter	Yüksek ✓	Şifre saklama
SHA512	sha512	128 karakter	Çok Yüksek ✓	Kritik veriler
HMAC	hmac_hesapla	64 karakter	Çok Yüksek ✓	API imzalama

Hızlı Başvuru Tablosu

Komut	Modül	Açıklama
<code>kullan sifrele</code>	—	Şifreleme modülünü aktive eder
<code>kullan hash</code>	—	Hash modülünü aktive eder
<code>b64_sifrele <d> <metin></code>	sifrele	Base64 kodlar
<code>b64_coz <d> <metin></code>	sifrele	Base64 çözer
<code>xor_sifrele <d> <metin> <anahtar></code>	sifrele	XOR+B64 şifreler
<code>xor_coz <d> <sifreli> <anahtar></code>	sifrele	XOR+B64 çözer
<code>dosya_sifrele "<kaynak>" "<hedef>" "<anahtar>"</code>	sifrele	Dosya şifreler
<code>dosya_coz "<kaynak>" "<hedef>" "<anahtar>"</code>	sifrele	Dosya çözer
<code>sifreli_yaz "<dosya>" "<metin>" "<anahtar>"</code>	sifrele	Şifreli yazar
<code>sifreli_oku <d> "<dosya>" "<anahtar>"</code>	sifrele	Şifreli okur
<code>sha256 <d> <metin></code>	hash	SHA-256 hash üretir
<code>sha512 <d> <metin></code>	hash	SHA-512 hash üretir
<code>md5 <d> <metin></code>	hash	MD5 hash üretir
<code>sha1 <d> <metin></code>	hash	SHA-1 hash üretir
<code>dosya_sha256 <d> "<dosya>"</code>	hash	Dosya SHA-256 hash
<code>hmac_hesapla <d> "<anahtar>" "<metin>"</code>	hash	HMAC-SHA256 üretir
<code>hash_dogrula <d> <metin> <hash> ["algo"]</code>	hash	Hash doğrular

Sonraki bölüm: `05_GUI_TKINTER.md` — Tkinter ile masaüstü pencere, buton, etiket ve form bileşenleri.